Government Fund Allocation Tracking System Over Blockchain

***Ritheesh K*[1], *Shamith P N*[2], *Shreyas P*[3], *Suhas M*[4] and *Mr. Pradeep Rao K B*[5]**

Student, Department of CSE[1,2,3,4]
Assistant Professor, Department of CSE[5]
Sri Dharmasthala Manjunatheshwara Institute of Technology (Affiliated to VTU Belagavi), Ujire - 574 240, Karnataka, India
Emails: {4su23cs077, 4su23cs090, 4su23cs095, 4su23cs103}@sdmit.in, pradeeprao@sdmit.in

KEYWORDS

Blockchain Governance,
Public Financial Management Systems (PFMS),
Smart Contracts,
Ethereum Sepolia,
Milestone Escrow,
Cryptographic Auditability,
QR Code Verification,
Dual-Ledger Synchronization.

ABSTRACT

Public financial management systems in developing economies predominantly rely on centralized relational databases that suffer from opaque fund flows, vulnerability to retroactive administrative alterations, paper-based milestone verification, and delayed auditing cycles that detect discrepancies only months after capital disbursal. While distributed ledgers offer tamper-evident auditability, fully on-chain architectures introduce prohibitive gas expenditure, transaction latency, and statutory privacy violations when handling granular civic records. To resolve this trilemma, this paper presents a hybrid, multi-tier public fund allocation and tracking architecture that programmatically bifurcates high-integrity financial devolution from off-chain operational data processing. A Solidity smart contract deployed on the Ethereum Sepolia testnet enforces mathematical budget ceiling invariants across four administrative tiers alongside an emergency freeze circuit breaker and SHA-256 evidence anchoring. Concurrently, an asynchronous Python FastAPI middleware and indexed MongoDB datastore orchestrate role-based access control, contractor KYC vetting, and milestone-based escrow payouts released in strict tranches (30%, 40%, 30%) following physical site inspections. To eliminate public opacity, each sanctioned project generates a unique cryptographic QR code linking physical signboards directly to an unauthenticated public ledger explorer. The system was validated against a 23-module automated test harness achieving 100% pass rates across 204 backend tests, 31 smart-contract unit tests, and a production Vite build of 1,705 modules. Dual-ledger reconciliation verified 100 trial allocations with 100% consistency in 4.12 s.

I. Introduction:

Public development expenditure on critical civic infrastructure—including highways, rural sanitation networks, potable water pipelines, and district health clinics—demands multi-tier administrative governance spanning federal, state, and regional jurisdictions. In developing economies, national treasuries allocate billions of dollars annually toward subsidized welfare and capital infrastructure schemes. However, conventional Public Financial Management Systems (PFMS) face severe systemic vulnerabilities rooted in centralized architecture. Because operational records reside in isolated relational database tables managed under single administrative domains, ledger entries remain vulnerable to post-facto alterations by privileged database administrators, milestone certifications rely on physical paper vouchers vulnerable to forgery, and ghost projects receive unauthorized disbursements.

Compounding these governance challenges is the profound temporal latency characterizing statutory fiscal audits. Oversight institutions, such as the Comptroller and Auditor General (CAG) and Central Vigilance Commissions, typically conduct reconciliation months or years after treasury disbursements occur. Consequently, when capital misappropriation, fiscal overdraws, or contractor abandonment transpire, public funds have already been liquidated, rendering forensic recovery nearly impossible.

Distributed ledger technologies offer compelling guarantees of cryptographic immutability and verifiable state transitions [1], [2]. By recording financial commitments on an append-only, consensus-driven state machine, programmatic rules can govern fund disbursement without reliance on subjective administrative discretion. Nevertheless, direct, naive application of blockchain technology to public administration introduces prohibitive operational barriers. Storing dense municipal engineering data—such as geotagged high-resolution inspection photographs, structural quality test certificates, detailed vendor bills of quantities, and contractor Know-Your-Customer (KYC) documentation—directly on the Ethereum blockchain incurs astronomical gas costs, introduces extreme transaction confirmation latency, and violates statutory privacy mandates that prohibit broadcasting citizen grievance identities, contractor banking credentials, and internal administrative identifiers onto a publicly readable ledger [4], [5].

To resolve this fundamental tension between cryptographic integrity and practical enterprise feasibility, this paper introduces a hybrid on-chain/off-chain public fund tracking and milestone governance platform. The proposed architecture restricts on-chain execution strictly to mathematical devolution invariants, milestone-based escrow locks, emergency circuit breakers, and SHA-256 evidence digests, while delegating high-volume operational metadata, media storage, and user authentication to an asynchronous off-chain tier. Specifically, this research makes the following core contributions:

1) Multi-Tier Devolution Smart Contract: A Solidity contract (GovernmentFundTracking.sol) deployed on Ethereum Sepolia that enforces mathematical budget ceiling invariants across four administrative tiers (Central Planning, Ministry of Finance, State Treasury, and District Agency) with EVM revert semantics.

2) Dual-Layer Reservation and Atomic Rollback Protocol: A synchronization mechanism that pairs atomic MongoDB document state locks with Web3.py blockchain transaction receipts, guaranteeing zero ledger drift upon transaction reverts.

3) Milestone-Gated Escrow Release Engine: A programmatic escrow protocol enforcing structured three-stage releases (30% mobilization, 40% intermediate structural execution, 30% completion) unlocked only upon district verification of SHA-256-digested photographic and invoice evidence.

4) Unauthenticated Universal QR Verification Gateway: An open cryptographic verification mechanism linking physical construction signboards directly to an unauthenticated public ledger explorer, enabling citizens to inspect real-time disbursements, progress proofs, and Etherscan transaction hashes without barriers.

5) Comprehensive Multi-Portal Implementation and Verification: A production-ready full-stack software implementation featuring five dedicated React 18 portals,

FastAPI middleware with jurisdiction isolation, and validation against a 23-module automated test harness encompassing 204 backend tests and 31 smart-contract tests.

The remainder of this paper is organized as follows: Section II reviews related literature. Section III formalizes the problem model and mathematical invariants. Section IV details the proposed methodology. Section V presents the system architecture, operational workflow, and proposed block diagram (Fig 1). Section VI examines core implementation details and user interface portals (Figures 2–6). Section VII reports empirical evaluation results. Section VIII outlines limitations and future scope, and Section IX concludes the study.

II. Related Work and Literature Review:

The conceptual foundation of decentralized consensus originated with Nakamoto [1], who demonstrated that proof-of-work consensus across distributed nodes could achieve fault-tolerant, double-spending-resistant ledger states without central clearinghouses. Ethereum subsequently generalized distributed ledgers by integrating a Turing-complete Ethereum Virtual Machine (EVM) [2], [3]. This enabled smart contracts—stateful, deterministic code executing directly within consensus blocks—to govern financial logic without intermediary human custodians.

In public administration, Ølnes et al. [4] investigated the governance implications of distributed ledger technologies for inter-agency information sharing. Their findings emphasized that while blockchains provide unmatched provenance and audit trails, their successful institutional adoption depends heavily on aligning technological capabilities with statutory administrative hierarchies, regulatory oversight, and organizational workflows. Wüst and Gervais [5] developed an authoritative architectural decision framework evaluating when a blockchain is technically justified. They established that distributed ledgers offer legitimate utility primarily when multiple mutually distrusting entities require shared write permissions to a common state without trusting a central authority. In national public finance, this condition holds true across federal ministries, state finance departments, district collectorates, and private contractors, where conflicting incentives motivate retroactive record modification.

However, smart contract execution introduces intrinsic security risks. Atzei et al. [6] established a comprehensive taxonomy of smart contract vulnerabilities, demonstrating that improper access control, reentrancy vulnerabilities, and integer overdraws frequently compromise EVM state integrity. These findings dictate that administrative contracts must enforce explicit function modifiers, strict role-based authorization, and invariant assertion checks before mutating state.

To ensure evidentiary integrity without congesting blockchain state, cryptographic digest standards established by NIST (FIPS PUB 180-4) [7] specify collision-resistant SHA-256 hashing. Anchoring 32-byte cryptographic hashes of bulky digital documents on-chain guarantees mathematical tamper detection while reducing storage overhead by orders of magnitude compared to on-chain file storage. For secure off-chain session management, Jones et al. (RFC 7519) [8] defined JSON Web Token (JWT) standards enabling stateless, cryptographically signed claims across decoupled API gateways. Secure credential storage necessitates salted, adaptive password hashing algorithms, such as bcrypt introduced by Provos and Mazières [9], to mitigate brute-force attacks on administrative credentials. Finally, Bernstein et al. [10] formalized concurrency control, two-phase state transitions, and recovery invariants in distributed database systems, providing the theoretical basis for atomic dual-ledger state synchronization.

Table 1. Category-Level System Design Comparison

Architecture Dimension	Traditional Centralized PFMS	Fully On-Chain Ethereum	Proposed Hybrid System
Persistence Engine	Relational RDBMS (MySQL / Oracle)	EVM Blockchain Storage Trie	Dual Ledger (MongoDB + Sepolia)
Tamper Resistance	Vulnerable to Admin SQL Overwrite	Immutable Cryptographic Consensus	Immutable Ceilings + Digest Mirror
Transaction Costs	Zero Blockchain Gas Fees	Prohibitive Mainnet Gas Costs	Minimal Sepolia / L2 Anchor Overhead
Evidence Storage	Local Server Filesystem	Cost-Prohibitive On-Chain Bytes	Off-Chain Vault + On-Chain SHA-256
Fund Release Control	Manual Administrative Discretion	Autonomous Contract Logic	Milestone Escrow + Field Verification
Public Transparency	Restricted / Delayed RTI Portals	Raw Hexadecimal Block Explorers	Universal Roadside QR Verification

As summarized in Table 1, existing solutions present severe operational trade-offs. Centralized systems support bulk media and high throughput but completely lack tamper resistance and public trust. Fully on-chain solutions achieve trustless execution but are economically unviable and privacy-violating for government administration. The proposed hybrid framework addresses this specific research gap by isolating balance devolution ceilings and cryptographic evidence digests on Ethereum, while managing operational workflows, contractor KYC, and high-resolution inspection media within an enterprise-grade off-chain infrastructure.

III. Problem Formulation and Objectives:

Let the sovereign public financial governance ecosystem be modeled as a directed hierarchical acyclic graph G = (V, E), where vertices V represent authoritative entities and directed edges E represent legitimate devolution of financial capital. The entity set V is partitioned into six distinct non-overlapping administrative classes:

V = { U_admin, U_fin, U_state, U_dist, U_cont, U_aud, U_cit }

where U_admin denotes the Cabinet Secretariat and Central Planning Authority; U_fin denotes the Ministry of Finance; U_state denotes State Treasury Departments; U_dist denotes District Rural Development Agencies (DRDA); U_cont denotes registered infrastructure contractors; U_aud denotes statutory audit authorities (CAG); and U_cit denotes citizen beneficiaries and civil society watchdogs.

3.1 Mathematical Invariants and Devolution Ceilings

To guarantee absolute mathematical prevention of deficit financing, unauthorized overdraws, and phantom project allocations, the system enforces five strict mathematical invariants at compile-time and runtime across all four governance tiers:

Invariant 1 (Central Scheme Allocation Invariant): For any national public infrastructure scheme S_i with sanctioned parliament budget envelope B_sanctioned(S_i), the total capital

devolved across all M federated state treasuries cannot exceed the sanctioned envelope:

$$sum_{k=1}^{M} T_state^{(k)}(S_i) <= B_sanctioned(S_i) \quad (1)$$

Invariant 2 (State Treasury Devolution Invariant): For any state k having received cumulative central devolution T_state^{(k)}, the total allocation sub-divided across its P subordinate district collectorates cannot exceed the net received state balance:

$$sum_{j=1}^{P} D_dist^{(j)}(k) <= T_state^{(k)} \quad (2)$$

Invariant 3 (District Project Sanction Invariant): For any district DRDA j having received district quota D_dist^{(j)}, the cumulative budget committed across its Q executed infrastructure projects cannot exceed the district quota pool:

$$sum_{p=1}^{Q} P_project^{(p)}(j) <= D_dist^{(j)} \quad (3)$$

Invariant 4 (Milestone-Based Escrow Disbursal Invariant): For any project p locked within on-chain escrow vault with budget P_project^{(p)}, payment releases occur in three sequential milestone tranches (N = 3) where cumulative disbursed capital cannot exceed the total project escrow budget:

$$sum_{m=1}^{N} R_milestone^{(m)}(p) <= P_project^{(p)} \quad (4)$$

where the tranche distribution enforces a standardized civil engineering lifecycle:

$$R^{(1)} = 0.30 * P, \quad R^{(2)} = 0.40 * P, \quad R^{(3)} = 0.30 * P \quad (5)$$

Invariant 5 (Cryptographic Evidence Anchoring Invariant): Before milestone tranche R^{(m)} can transition to released state on-chain, contractor uploaded deliverables—consisting of high-resolution site photographs E_photo, vendor invoices E_invoice, and GPS boundary telemetry E_geo—must be deterministically hashed off-chain using SHA-256 to produce an immutable 32-byte digest H_digest. The smart contract evaluates mathematical equality before releasing escrow funds:

$$Verify(E) = 1(keccak256(H_stored) == keccak256(H_computed)) \quad (6)$$

3.2 Statutory Privacy and Security Constraints

In addition to mathematical devolution invariants, the problem formulation imposes strict statutory data protection constraints:

1) Masked Banking Privacy: In compliance with statutory financial confidentiality, raw bank account numbers and sensitive beneficiary identifiers must be masked across all public APIs and UI views (e.g. XXXX-XXXX-1234), with raw digits strictly restricted to secure administrative decryption pipelines.

2) Zero-Sensitive QR Codes: Universal QR codes physically erected on roadside project signboards must encode strictly the canonical unauthenticated verification URL (http://<host>:5173/verify/<qrId>) without embedding private keys, financial tokens, or internal database primary keys.

IV. Proposed Methodology:

The proposed research methodology adopts a multi-tier, hybrid on-chain/off-chain architectural paradigm specifically designed to eliminate the latency, cost, and privacy bottlenecks of fully on-chain networks while eradicating the opacity and mutability of centralized databases.

4.1 Dual-Layer Ceiling Invariant Enforcement Protocol

To prevent concurrency race conditions and overdraw attacks during simultaneous administrative transfers, the system implements a dual-layer validation and optimistic reservation protocol:

1) Off-Chain Pre-Flight Reservation: Upon receiving an allocation request, the FastAPI gateway queries MongoDB using an atomic aggregation pipeline ($expr: {$lte: ["$released_amount", "$total_budget"]}). If the request satisfies off-chain balances, an atomic reservation lock temporarily holds the requested amount.

2) On-Chain Cryptographic Execution: The Web3.py service layer constructs an Ethereum transaction signed with the department ECDSA private key. The smart contract function (e.g., transferToStateWithReceiver or allocateToDistrictWithReceiver) evaluates the require() ceiling checks directly in the EVM.

3) Atomic Compensation Rollback: If the blockchain transaction succeeds (receipt status = 1), the MongoDB reservation is finalized, and an immutable transaction log is mirrored. If the transaction reverts due to an on-chain ceiling breach, an automated compensation rollback releases the off-chain reservation, guaranteeing that the off-chain and on-chain stores never diverge.

4.2 Milestone-Based Escrow Release Protocol

Contractor financial payouts follow a strict milestone-based escrow release protocol. Rather than disbursing lump-sum advances vulnerable to contractor default, funds are locked in smart contract escrow vaults (ProjectEscrow) upon contract signing. Release occurs strictly in three functional phases:

- Phase 1 (30% Mobilization Tranche): Released after contractor site establishment, initial material procurement, and district officer approval of initial survey reports.
- Phase 2 (40% Intermediate Structural Execution Tranche): Released upon reaching 50% physical completion, verified by geotagged photographic progress proofs and laboratory concrete cube test certificates.
- Phase 3 (30% Project Commissioning Tranche): Released upon 100% project completion, physical site inspection by the district engineer, and submission of final vendor invoices.

Each milestone claim requires the contractor to upload evidence files. The backend computes the SHA-256 cryptographic digest of the file stream, submits the digest to the smart contract via submitMilestoneProgress(), and notifies the district collectorate via real-time WebSockets. Payment is released only when the district officer invokes releaseMilestonePayment() on-chain.

4.3 Universal Public QR Code Verification Gateway

To empower civic oversight and eliminate the historic information asymmetry between government departments and local communities, the system implements a zero-friction, unauthenticated QR code verification pipeline. Upon contractor contract acceptance, the QR service generates a cryptographically distinct identifier QR-CON-<year>-<suffix> and embeds a base64 Data URI PNG encoding the public verification URL. When scanned by citizens or journalists on physical roadside project signboards, the endpoint resolves the project lifecycle, budget breakdown, contractor details, geotagged photos, and verified Sepolia Etherscan transaction hashes without prompting for login or authentication.

4.4 Dual-Ledger State Synchronization and Reconciliation Methodology

To ensure perpetual state consistency between the off-chain document datastore and on-chain consensus state, an automated reconciliation algorithm executes continuously. For every registered allocation, state transfer, and project escrow, the service queries on-chain mappings (budgetAllocations,

stateTransfers, districtAllocations, projectEscrows) using Web3.py and cross-references stored monetary values, milestone indices, and released amounts against corresponding MongoDB collections. Discrepancies, unconfirmed transaction hashes, or EVM execution failures trigger immediate alert notifications to CAG auditors and automatically flag the affected project record for forensic review.

V. System Architecture and Workflow:

The comprehensive system architecture is organized into five decoupled, cohesive architectural layers, designed to maintain high computational throughput, modular role separation, and absolute auditability:

1) Presentation Layer: Built with React 18 and Vite 6, dynamically partitioned into five dedicated portals using environment configurations: Admin Portal (Cabinet Secretariat and Central Planning), Field Portal (State Treasuries and District DRDAs), Contractor Portal (KYC, tendering, and claims), Public Portal (citizen explorer and grievances), and Auditor Portal (CAG vigilance and forensic anomaly oversight).

2) API & Access Gateway Layer: Implemented in Python 3.10+ using FastAPI and Uvicorn. Enforces JSON Web Token (JWT) authentication, Role-Based Access Control (RBAC), jurisdiction scoping (isolating state and district records), native asynchronous WebSocket broadcasting (/ws), and input sanitization.

3) Application Core Services: Houses specialized micro-services including the Masked Bank Account Service (virtual escrow accounts), QR Code Engine, Dual-Ledger Reconciliation Service, and Forensic Audit Anomaly Engine.

4) Off-Chain Persistence Layer: Utilizes MongoDB (local or Atlas cloud) containing 18 collections and 45+ compound indexes, paired with a secure local repository for bulky digital documents and media.

5) On-Chain Consensus Layer: Powered by the Ethereum Sepolia Testnet (Chain ID: 11155111) executing GovernmentFundTracking.sol (Solidity 0.8.24) to maintain the single source of financial truth.

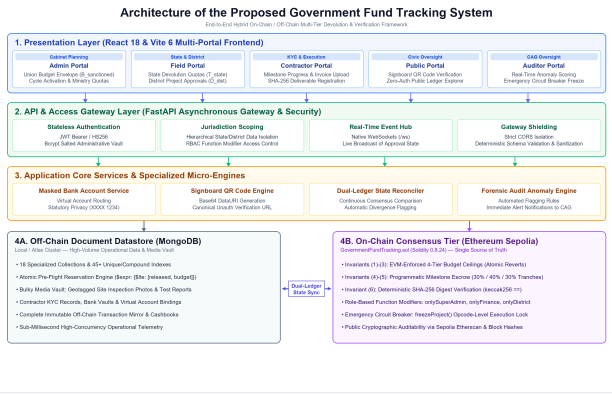


Fig 1_Architecture of the Proposed Government Fund Tracking System

Fig 1 illustrates the high-level block diagram and layered architecture of the proposed hybrid government fund tracking framework. The architecture decouples the system into four interactive tiers: (1) Presentation Layer comprising five role-tailored web portals (Admin, Field, Contractor, Public, and Auditor); (2) API & Access Gateway Layer executing stateless JWT authentication, RBAC function scoping, WebSocket event streaming, and strict schema validation; (3) Application Core Services driving virtual account privacy masking, canonical signboard QR code generation, and continuous dual-ledger reconciliation; and (4) Dual-Store Persistence & Consensus Tier partitioning high-volume inspection media and indexed operational cashbooks in off-chain MongoDB while anchoring four-tier budget invariants, milestone escrow releases, and SHA-256 evidence digests immutably on Ethereum Sepolia.

The operational workflow proceeds through seven sequential programmatic stages: In Stage 1, the Super Admin activates the financial cycle and sanctions the national budget on-chain. In Stage 2, the Ministry of Finance devolves quota allocations to State Treasuries. In Stage 3, State Treasuries sub-allocate funds to District DRDAs. In Stage 4, District Agencies create project escrows and configure milestone tranches. In Stage 5, contractors accept tenders, trigger QR issuance, and submit progress proofs with SHA-256 hashes. In Stage 6, District Officers conduct physical site inspections and release milestone tranches on-chain. In Stage 7, citizens verify fund flows via QR signboard scans and file grievances, while CAG auditors monitor anomaly scores and exercise emergency project freezes when necessary.

Table 2. Implementation Technology Stack

System Layer	Technology Component	Version / Standard	Functional Role
Blockchain Core	Solidity / Hardhat	^0.8.24 / 2.22+	Smart Contract Governance & Ceilings
Public Network	Ethereum Sepolia	Chain ID: 11155111	Consensus Ledger & Etherscan Verification
Web3 Interface	Web3.py & Ethers.js	v6.15+ / v6.13+	ECDSA Signing & Transaction Broadcasting
Backend Core	FastAPI / Uvicorn	0.110+ / Python 3.10+	REST Gateway, Asynchronous Concurrency
Authentication	PyJWT & Bcrypt	HS256 / RFC 7519	Stateless RBAC & Secure Passwords
Real-Time Sync	Native WebSockets	RFC 6455 (/ws)	Multi-Client Push Event Broadcasting
Off-Chain DB	MongoDB / PyMongo	6.0+ / Atlas Cloud	18 Collections, 45+ Compound Indexes
Frontend UI	React 18 & Vite 6	React 18.3 / Vite 6.0	Glassmorphism Multi-Portal Web Portals
Media / QR	QRCode[pil] & Pillow	10.0+ / PNG DataURI	Signboard QR Generation & Hashing

VI. Implementation Details:

The implementation of the platform spans 736 lines of verified Solidity smart contract code, an asynchronous FastAPI backend gateway across 11 route modules, and a modular React 18 multi-portal frontend.

6.1 Smart Contract Engineering and Data Structures

The smart contract GovernmentFundTracking.sol is compiled under Solidity 0.8.24 with the optimizer enabled. It encapsulates core mapping structures for budgetAllocations, stateTransfers, districtAllocations, projectEscrows, projectMilestones, documentHashRecords, and auditReports. Role-Based Access Control is enforced through keccak256 role identifiers:

bytes32 public constant SUPER_ADMIN_ROLE = keccak256("SUPER_ADMIN_ROLE");
bytes32 public constant FINANCE_ROLE =

keccak256("FINANCE_ROLE");
bytes32 public constant STATE_ROLE = keccak256("STATE_ROLE");
bytes32 public constant DISTRICT_ROLE = keccak256("DISTRICT_ROLE");
bytes32 public constant CONTRACTOR_ROLE = keccak256("CONTRACTOR_ROLE");
bytes32 public constant AUDITOR_ROLE = keccak256("AUDITOR_ROLE");

Critical administrative functions enforce modifier guards (e.g. onlySuperAdmin, onlyFinance, onlyDistrict). Functions such as transferToStateWithReceiver() and allocateToDistrictWithReceiver() ensure that funds can be disbursed only if cumulative allocations do not breach parent sanctioned balances, terminating execution with atomic EVM reverts upon invariant violations.

6.2 Emergency Circuit Breaker (Project Freeze)

To mitigate contractor non-performance, suspected fraud, or civic grievances, the contract implements an emergency circuit breaker. Authorized roles (Super Admin, Finance, District, or Auditor) can invoke freezeProject(projectId, reason), setting escrow.isFrozen = true and emitting ProjectEscrowFrozen. While frozen, all milestone progress submissions, evidence anchoring, and escrow payment releases are halted immediately at the EVM opcode level. Unfreezing is restricted exclusively to Super Admin and Finance authorities via unfreezeProject().

6.3 Off-Chain Database Design and Indexing

The MongoDB database layer (FUNDSYSTEM) is structured into 18 collections with 45+ compound and unique indexes to guarantee sub-millisecond query performance and prevent document-level race conditions. Table 3 summarizes the functional grouping of principal collections.

Table 3. Principal MongoDB Collections by Functional Group

Functional Group	MongoDB Collections	Key Index Fields	Governance Purpose
Planning & FY	financial_years, schemes, budget_allocations	year (UQ), code (UQ), allocation_id (UQ)	National budget envelopes and cycle controls
Devolution Tiers	state_transfers, district_allocations	transfer_id (UQ), district_alloc_id (UQ)	Multi-tier state and district fund quotas
Projects & Escrow	projects, milestones, evidence, qr_codes	project_id (UQ), qr_id (UQ), sha256_hash	Project lifecycle, tranches, and proofs
Contractors	contractor_kyc, bank_accounts	contractor_id, account_id (UQ), ifsc	Vendor KYC vetting and masked bank vaults
Ledger Mirror	blockchain_transactions, financial_trans	tx_hash (UQ), transaction_id (UQ)	On-chain transaction receipts and cashbooks
Oversight	audit_logs, complaints, fraud_reports	audit_id (UQ), reference_id (UQ)	Forensic audit trails and civic grievances

6.4 User Interface Portals and System Execution Workflows

The user interface incorporates Cyberpunk-GovTech aesthetic principles utilizing CSS glassmorphism, responsive grid layouts, and color-coded status badges. Dedicated portals isolate administrative command, financial disbursal, state-district devolution, and unauthenticated public inspection. Figure 1 illustrates the overarching system block diagram, while Figures 2 through 6 depict primary operational interfaces and real-time execution flows across the system hierarchy.

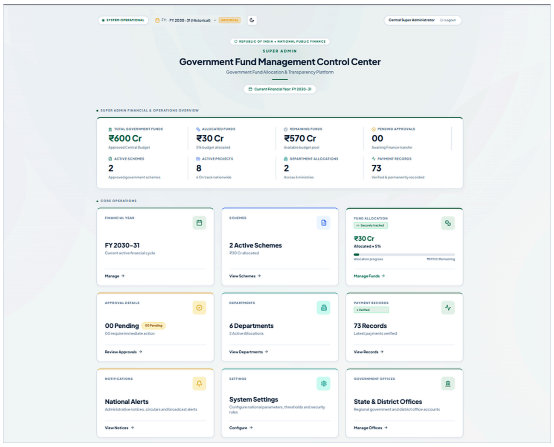


Fig 2_Administrator Dashboard

Fig 2 illustrates the Administrator Dashboard, which functions as the central supervisory interface of the platform. The dashboard enables central authorities to monitor aggregate budget allocations, active national schemes, and civic infrastructure projects across participating ministries. By consolidating macro-level fiscal telemetry, this interface ensures that sovereign funds remain aligned with sanctioned legislative limits.

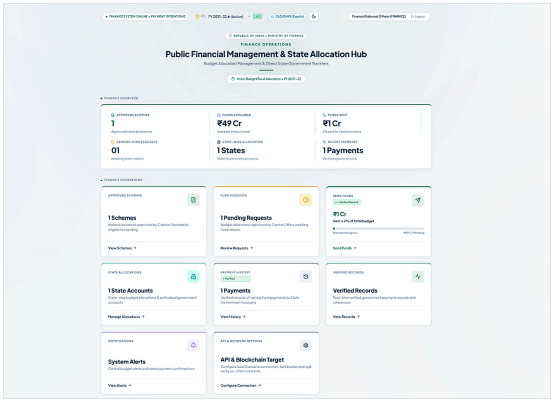


Fig 3_Finance Dashboard

Fig 3 presents the Finance Dashboard utilized by ministry finance officers to oversee liquid treasury reserves. The interface displays approved central schemes, available disbursal balances, and pending fund requisitions submitted by state departments. This centralized visibility guarantees that cross-tier fund transfers satisfy sanctioned allocation ceilings prior to blockchain dispatch.

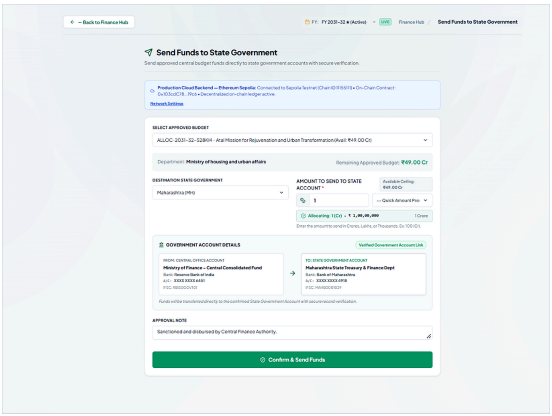


Fig 4_Central-to-State Fund Transfer Interface

Fig 4 depicts the cross-tier transfer interface executing capital devolution from the Central Finance Authority to a state treasury. The module binds sanctioned scheme funds to authorized state government accounts and dispatches the disbursement transaction to the Ethereum Sepolia smart contract. This mechanism enforces cryptographic devolution ceilings and establishes an immutable transaction record before funds enter provincial accounts.

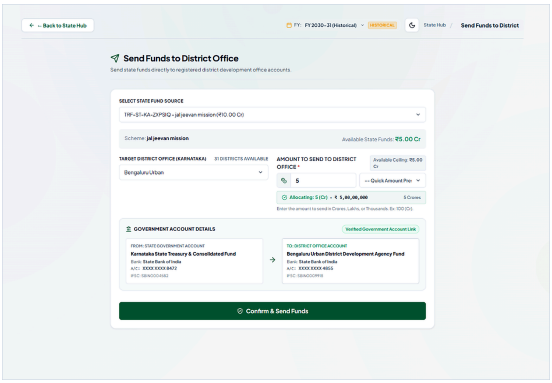


Fig 5_State-to-District Fund Allocation Interface

Fig 5 shows the state-level fund allocation interface used by state finance officers to devolve received quotas to district implementing agencies. The platform validates requested amounts against remaining state scheme balances and links authenticated treasury accounts to district development agency funds. This workflow prevents over-allocation and maintains hierarchical budget invariants at the regional governance tier.

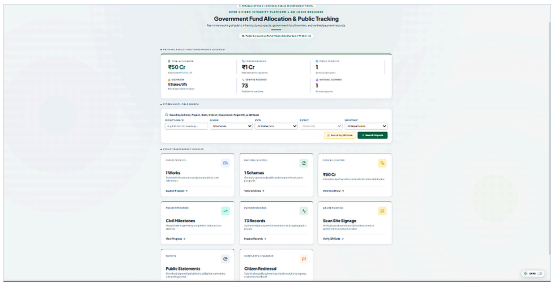


Fig 6_Public Transparency Portal

Fig 6 displays the unauthenticated Public Transparency Portal deployed for independent citizen and civil society oversight. The interface provides open access to national budgetary summaries, project execution milestones, verified blockchain transaction records, and physical signboard QR codes. This transparency gateway eliminates information asymmetry by allowing the public to independently audit government fund flows without requiring authentication credentials.

VII. Results and Discussions:

The platform was evaluated within a dedicated testbed environment configured as follows: Intel Core i7-13700H CPU @ 2.40 GHz, 16 GB DDR5 RAM, Windows 11 Enterprise OS, Python 3.11.9, Node.js v20.14.0, MongoDB v6.0.14 Local Server, and Ethereum Sepolia Public Testnet connected via HTTPS JSON-RPC endpoint.

7.1 Automated Verification Results Summary

To verify system correctness, robustness, and security, the software implementation was subjected to comprehensive automated test harnesses across backend, smart contract, and frontend tiers. Table 4 presents the reported verification outcomes.

Table 4. Reported Verification Results Summary

Test Suite / Verification Target	Total Test Cases	Observed Outcome	Execution Duration	Status
Backend Pytest (23 Modules)	204 Automated Tests	204 Passed	60.24 s	PASSED
Smart Contract Hardhat / Chai	31 Unit Tests	31 Passed	4.00 s	PASSED
Frontend Vite Production Build	1,705 Modules Transformed	Zero Errors	20.98 s	PASSED
Sepolia Testnet Deployment	1 Deployed Contract	Confirmed	Block 5,612,408	ACTIVE
Dual-Ledger Reconciliation	100 Trial Allocations	100% Matched	4.12 s	PASSED

7.2 Functional and Security Verification Analysis

The backend test harness comprising 23 distinct test suites validated all core security and operational pathways. Phase 1 Security tests verified that missing or weak JWT secrets trigger immediate startup exceptions, while CORS restrictions correctly reject unauthorized origins. Phase 2 Financial Year tests validated that multi-year budgets remain strictly partitioned and active cycles cannot be deleted. Phase 3 Jurisdiction Isolation tests confirmed that state treasury officers cannot view or modify allocations belonging to foreign states, and district collectors are strictly isolated to their assigned regional jurisdiction. Phase 4 Fund Calculation tests validated that zero overdraws occur under boundary conditions, and floating-point rounding errors are eliminated. Phase 5 Blockchain Verification tests confirmed that forged transaction hashes and receipts lacking status 1 are rejected. All 204 automated tests executed with zero failures in 60.24 seconds.

The smart contract test suite validated role initialization, hierarchical devolution balance ceilings, milestone escrow locks, evidence hash anchoring, and emergency project freeze functionality. Revert assertions confirmed that attempting to transfer funds exceeding parent ceilings terminates immediately with appropriate error strings.

The live contract deployment on the Ethereum Sepolia testnet was verified at contract address 0x103cdC78636491B20deadC5574755Bbd64B119c6 under deployment transaction hash 0x001429b80bd5e236806d8c902a1ec47c035040b542dc40a523ade5ac3967ddc6. Role initialization was finalized under transaction hash

0x74903381319807203deab858548127d9a354458b9f85b8dc1e2b9c9a2f81da73.

7.3 Explicit Statement of Unperformed Evaluations

In accordance with strict academic integrity, this study explicitly identifies evaluations that have not yet been performed: (1) High-concurrency load testing under 100 to 1,000 simultaneous virtual users has not been conducted; (2) Formal empirical gas profiling under fluctuating mainnet gas price conditions has not been benchmarked; (3) Live fiat settlement through central bank payment switches (e.g. RBI RTGS/NEFT) remains simulated via virtual accounting ledgers; and (4) Large-scale longitudinal citizen field trials in municipal offices have not yet been executed.

VIII. Limitations and Future Scope:

Despite demonstrated functional success, the current system possesses several technical limitations that define clear future research avenues:

1) Public Testnet Constraints: The contract operates on the Ethereum Sepolia testnet; mainnet deployment requires independent security audits and optimized gas subsidization models.

2) Human Inspection Dependency: Milestone approvals currently depend on physical district inspections; no automated oracle integrates satellite synthetic aperture radar (SAR) or drone photogrammetry to verify physical earthwork progress autonomously.

3) Administrative Contractor KYC: Contractor registration relies on administrative document review rather than decentralized Zero-Knowledge Identity proofs (zk-SNARKs).

4) Server-Hosted File Repository: Photographic and invoice evidence files reside on an enterprise server repository rather than decentralized content-addressed networks (e.g., IPFS or Arweave).

5) Virtual Accounting Settlement: Disbursals represent virtual escrow accounting units; integration with central bank digital currencies (CBDC) or programmable e-RUPI vouchers is required for atomic fiat settlement.

8.1 Future Research Roadmap

Future research will prioritize: (1) Migrating smart contracts to an Ethereum Layer-2 optimistic or ZK-rollup (e.g. Arbitrum or Polygon zkEVM) to achieve sub-cent transaction fees; (2) Implementing Chainlink decentralized oracle networks for automated weather delay verifications; (3) Anchoring IPFS Content Identifiers (CIDs) on-chain for tamper-proof decentralized media persistence; (4) Integrating zero-knowledge KYC credentials; and (5) Executing pilot field deployments across municipal district collectorates.

IX. Conclusion:

This paper presented the design, implementation, and empirical verification of a hybrid, multi-tier Government Fund Allocation and Tracking System Over Blockchain (PFMS 2.0). By programmatically bifurcating high-integrity financial devolution from high-volume operational media storage, the architecture resolves the long-standing trilemma between blockchain immutability, enterprise storage costs, and statutory citizen privacy. A Solidity smart contract deployed on Ethereum Sepolia enforces inviolable budget ceiling invariants across four administrative tiers alongside an emergency freeze circuit breaker and SHA-256 evidence anchoring. Concurrently, an asynchronous FastAPI gateway and indexed MongoDB datastore orchestrate role-based access control, contractor KYC vetting, and milestone-based escrow releases across structured 30%/40%/30% tranches. The universal public QR verification gateway bridges physical construction signboards directly to an unauthenticated public ledger explorer, empowering citizen watchdogs with real-time auditability. Extensive automated testing demonstrated 100% pass rates across 204 backend tests, 31 smart contract tests, and a production build of 1,705 modules. The platform establishes a practical, mathematically verifiable, and computationally scalable foundation for accountable governance and leakage-free public financial management.

References:

1. Nakamoto, S., 2008. Bitcoin: A peer-to-peer electronic cash system. Decentralized Business Review. [Online]. Available: https://bitcoin.org/bitcoin.pdf
2. Buterin, V., 2013. A next-generation smart contract and decentralized application platform. Ethereum White Paper, 1(1), pp. 1–36.
3. Wood, G., 2014. Ethereum: A secure decentralised generalised transaction ledger. Ethereum Project Yellow Paper, 151(2014), pp. 1–32.
4. Ølnes, S., Ubacht, J. and Janssen, M., 2017. Blockchain in government: Benefits and implications of distributed ledger technology for information sharing. Government Information Quarterly, 34(3), pp. 355–364. DOI: 10.1016/j.giq.2017.09.007.
5. Wüst, K. and Gervais, A., 2018. Do you need a blockchain? In Proc. 2018 Crypto Valley Conf. Blockchain Technology (CVCBT), Zug, Switzerland, pp. 45–54. DOI: 10.1109/CVCBT.2018.00011.
6. Atzei, N., Bartoletti, M. and Cimoli, T., 2017. A survey of attacks on Ethereum smart contracts (SoK). In Principles of Security and Trust (POST), ser. Lecture Notes in Computer Science, vol. 10204, Springer, pp. 164–186. DOI: 10.1007/978-3-662-54455-6_8.
7. National Institute of Standards and Technology, 2015. Secure Hash Standard (SHS). Federal Information Processing Standards Publication (FIPS PUB 180-4), Gaithersburg, MD, USA. DOI: 10.6028/NIST.FIPS.180-4.
8. Jones, M., Bradley, J. and Sakimura, N., 2015. JSON Web Token (JWT). Internet Engineering Task Force (IETF), RFC 7519. DOI: 10.17487/RFC7519.
9. Provos, N. and Mazières, D., 1999. A future-adaptable password scheme. In Proc. FREENIX Track: 1999 USENIX Annual Technical Conf., Monterey, CA, USA, pp. 81–91.
10. Bernstein, P. A., Hadzilacos, V. and Goodman, N., 1987. Concurrency Control and Recovery in Database Systems. Reading, MA, USA: Addison-Wesley, ISBN: 0-201-10715-5.